

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedy vs Dinámico vs Bruto.»

Isidora Sofía Villegas Hernández

22 de junio de 2026

22:40

1. Implementaciones

2. Implementaciones

En esta sección se describen los algoritmos implementados para resolver el problema de optimización de satisfacción en el visionado de animes bajo restricciones de tiempo y energía.

2.1. Algoritmo de Fuerza Bruta

Se implementó una búsqueda exhaustiva mediante **Backtracking**. El algoritmo explora recursivamente todas las combinaciones posibles de prefijos para cada anime.

- **Lógica:** Para cada anime i , se prueba con no ver ningún capítulo o ver un prefijo de longitud $k \in \{1, \dots, q_i\}$. Si los recursos (tiempo M y energía E) no se agotan, se procede al siguiente anime.
- **Complejidad Temporal:** $O(\prod_{i=1}^n (q_i + 1))$. Debido a su crecimiento exponencial, este algoritmo solo es viable para instancias muy pequeñas ($n < 15$).
- **Complejidad Espacial:** $O(n)$, correspondiente a la profundidad máxima del árbol de recursión.

2.2. Programación Dinámica

Se modeló el problema como una variante del **Problema de la Mochila Multidimensional y de Múltiple Opción** (MMKP).

- **Estado:** $DP[m][e]$ representa la máxima satisfacción obtenible con exactamente m minutos y e unidades de energía.

- **Transición:** Para cada anime, se actualiza el estado considerando cada prefijo posible de manera excluyente (solo se puede elegir un prefijo por anime). Para evitar usar el mismo anime varias veces en una misma iteración, se utiliza una copia temporal del estado previo.
- **Complejidad Temporal:** $O(n \cdot M \cdot E \cdot \max(q_i))$. Con $n = 200, M = 3000, E = 500, q_i = 30$, el total de operaciones es manejable en tiempos modernos ($\approx 9 \times 10^9$ operaciones teóricas, pero mucho menos en la práctica por las restricciones de recursos).
- **Complejidad Espacial:** $O(M \cdot E)$ mediante la técnica de optimización de espacio.

2.3. Heurísticas Greedy

Se implementaron dos enfoques voraces con criterios locales distintos:

2.3.1. Greedy 1: Siguiendo Capítulo

Este algoritmo avanza capítulo a capítulo". En cada paso, evalúa el siguiente capítulo disponible de todos los animes.

- **Criterio:** Maximizar el ratio $v_{i,j} / (t_{i,j} + c_{i,j})$. Si el capítulo completa el anime, se suma el bono b_i a la satisfacción antes de calcular el ratio.
- **Complejidad:** $O(Q \cdot n)$, donde Q es la cantidad total de capítulos.

2.3.2. Greedy 2: Mejor Prefijo Global

A diferencia del anterior, este evalúa todos los prefijos posibles de los animes restantes en cada paso.

- **Criterio:** Seleccionar el prefijo de capítulos $(1 \dots k)$ que maximice el ratio de satisfacción acumulada sobre el costo total acumulado: $(\sum v + b) / (\sum t + \sum c)$.
- **Complejidad:** $O(n^2 \cdot \max(q_i))$.

3. Entorno experimental

4. Introducción Experimental

El experimento consiste en evaluar el rendimiento de los cuatro algoritmos implementados bajo diferentes escenarios. Se generaron 27 casos de prueba divididos en tres categorías:

- **Casos Pequeños** ($n \in \{3, 5, 8\}$): Diseñados para validar que la Fuerza Bruta y la Programación Dinámica coinciden en el resultado óptimo.
- **Casos Medianos** ($n \in \{20, 40, 80\}$): Orientados a medir la brecha de calidad entre las heurísticas Greedy y la solución óptima de DP.

- **Casos Grandes** ($n \in \{100, 150, 200\}$): Utilizados para analizar el escalamiento temporal y de memoria de los algoritmos eficientes.

Las pruebas se ejecutaron en una máquina con sistema operativo Linux, midiendo el tiempo de ejecución en milisegundos mediante la librería `chrono` y el uso de memoria máximo (`ru_maxrss`) mediante `getrusage`.

5. Resultados y Análisis

6. Resultados Experimentales

A continuación se presentan los resultados obtenidos tras ejecutar la suite de pruebas.

6.1. Tiempo de Ejecución

La Figura 1 muestra el tiempo de ejecución en escala logarítmica respecto al número de animes n .

Figura 1: Tiempo de ejecución promedio por algoritmo.

Se observa que la Programación Dinámica crece de manera lineal respecto a n , lo cual es consistente con su complejidad teórica $O(n \cdot M \cdot E \cdot q)$. Las heurísticas Greedy son órdenes de magnitud más rápidas, manteniendo tiempos casi instantáneos incluso para $n = 200$.

6.2. Uso de Memoria

La Figura 2 presenta el consumo de memoria máximo durante la ejecución.

Figura 2: Uso de memoria máximo por algoritmo.

El uso de memoria de DP se mantiene constante una vez que la matriz de 3000×500 es reservada. Las variaciones menores observadas en los gráficos corresponden a las estructuras de datos que almacenan la instancia del problema.

6.3. Calidad de la Solución

La calidad se define como el ratio entre la satisfacción de Greedy y la de DP (óptima).

Figura 3: Ratio de calidad de las heurísticas Greedy.

Se observa que **Greedy 2** (Mejor Prefijo Global) tiende a ofrecer mejores resultados que **Greedy 1** (Siguiendo Capítulo), ya que considera el beneficio de completar un anime (bono) de manera más integral al evaluar prefijos completos.

7. Conclusiones

8. Conclusiones

Tras el análisis experimental de los cuatro algoritmos propuestos para el problema AniMaraton, se concluye lo siguiente:

1. **Optimalidad vs Eficiencia:** El algoritmo de Fuerza Bruta es impráctico para cualquier instancia real, confirmando la necesidad de enfoques más sofisticados. La Programación Dinámica logra la solución óptima en un tiempo razonable ($\approx 1,3$ segundos para $n = 200$), lo que la convierte en la opción preferida si la exactitud es crítica.
2. **Desempeño de Heurísticas:** Las heurísticas Greedy, aunque no garantizan el óptimo, demostraron ser extremadamente rápidas. **Greedy 2** se destaca por capturar mejor la estructura del problema al evaluar prefijos, logrando una calidad de solución superior a Greedy 1 en la mayoría de los casos.
3. **Impacto de los Bonos:** El bono de completación es un factor determinante. Los algoritmos que "miran hacia adelante" (DP y Greedy 2) logran aprovechar mejor estos bonos en comparación con enfoques más miopes (Greedy 1).
4. **Uso de Recursos:** El uso de memoria está dominado por la tabla DP. En sistemas con memoria muy restringida, una heurística Greedy sería la única alternativa viable.

En conclusión, para el problema AniMaraton, la Programación Dinámica es altamente efectiva dadas las restricciones actuales, mientras que Greedy 2 ofrece un excelente balance para casos donde el tiempo de respuesta deba ser milimétrico.